

creating tables

DELETING

address with max qty above avg
WITH cte AS (S) and-id, Max (quantity) as max_order, (Avg Quantity) as avg_order
(S) and-id (F) and-id (W) max-end > (S) Max (avg order) Elde

friend request: has most friends

product at given price

$S * F \text{ Prod } LJ((S \text{ prod-id, new price } F) \text{ Prod } W) (\text{prod, chg date}) IN$

customer target num of orders

$$\frac{(\text{S cust num}) (\text{Orders}) (\text{GB}) \text{ cust num} (\text{H}) (\text{C}) *}{(\text{F}) (\text{Orders}) (\text{GB}) \text{ cust num}} \text{ AS counts-per-cust};$$

Student num in dep
WITH cl AS (S d.* s.stid(f) DEP d(LJ) St d s(ON) dep d) B dep-no

Tree Nodes

⑤ id, CASE W p -id IS NULL T "Root" WHEN id NOT IN (S id) F T

[E] A. follower as foll^o, A. followed by no num [E] [G] follower, [C] * as follow
 [F] follow [GB] follower, A [INNER] [S] follower, [C] * AS follows [F] follow

WITH the AS (SELECT u.userid, SUM(CASE WHEN year(o.ordate)=2019 THEN
as ord2019) F User u (U) Ord o ON u.userid = o.buyerid (G-B userid)

③ ① page id as recop page f likes as ② ③ ④ * ⑤ Friendship w uo1 i
 en uo2 id = 1) T ⑥ ⑦ l. uo id = T uo1 id OR l. uo id = T. uo2 id ⑧ l. pg id
 NOT IN ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳ ㉑ ㉒ ㉓ ㉔ ㉕ ㉖ ㉗ ㉘ ㉙ ㉚ ㉛ ㉜ ㉝ ㉞ ㉟ ㊱ ㊲ ㊳ ㊴ ㊵ ㊶ ㊷ ㊸ ㊹ ㊺ ㊻ ㊼ ㊽ ㊾ ㊿

[3] e.%, CASEW op = '>' T (IF (v1.val > v2.val) = 0, 'false', 'true') etc
 ELSE NULL END AS value [F] Exp e [L] Var v1 [ON] e. lrf op = v1 name
 [L] Var v2 [ON] e. rlf op = v2 name

UNION ALL | S.userid, credit | F.userid | S.t.userid, u.user name, sum (t. as credit, CASE WHEN sum (t. credit) < 0 Then 'yes' else 'no' END as

WITH T (sal, nb) AS ($\boxed{3}$ e1.sal, $\boxed{4}$ e2.nb $\boxed{5}$ Emp as e1)

client j'aurais commandé

[5] dep name as dep, emp name as emp, salary [F] Emp [J]
 [5N] emp dep id = dep id [W] (dep, salary) IN (5) dep id

[S] seller-id [F] sales [GB] sellerid [H] sum(price) = ([S] max
[F] ([S] seller-id, sum(price) total-price [F] sales [GB] sellerid

$\{S\} \text{ buyer_id } \{F\} \text{ Sales } \{W\} \text{ prod_id } = 1 \text{ EXCEPT } (\{S\} \text{ buyer_id } \{F\} \text{ Sales } \{W\} \text{ prod_id } = 3)$

Q11 Write a query to find the product name and price of the product whose price is greater than 0 and the product name is 'iPhone'.

Def: $\overline{GB} \text{ id}$

Relat $x1$ [LJ] Relat $x2$ [ON] $x1.\text{word} < x2.\text{word}$
 NP $x1.\text{full-id} = x2.\text{full-id}$, cte2 as ([S] word id ,

$\text{Mose}(\text{maxComm}) \text{ (f) } \text{cte2}$

"room" : —
 "age" : —

H. Join = Join on (1)
 and from = 3/5/5/5

X-Label " $\frac{3}{2}$ " :

"required": ["date", "dep"]

"X-concepts" et "x-refers to"

```
"properties": {  
  "date": {
```

si assegna

exit put; move

So in all
result

Scam
make is not now

scan backward

$\$0 = 1$ *du max?*

19-10

product price for each store/reformulate table month

product-id (CASE WHEN store = 'store1' THEN price END)
store1, etc idem
Products GB product id
most Req ordered prod for each customer
O2.prod-id, O2.cust-id COUNT(*) as order_count F Orders
AS O2
O2.prod-id, O2.cust-id H COUNT(*) =
MAX(prod-c) F (L) C(*) AS prod-count F orders as O1
O1.cust-id = O2.cust-id GB O1.prod-id as M ORDER BY

activity participants
acti, C(*) F Friends GB acti H C(*) = (L) max(c) F
F Acti F Friends ON Acti, name = friends.acti GP acti as M)
AND C(*) != (L) min(c) F (L) acti C* as c F GB as M)
give report (in) directly to boss
a.emp-id F Emp as a F Emp as b F Emp as c ON a.man-id
= b.emp-id AND b.man-id = c.emp-id W c.man-id = 1 AND a.em
-id = 1
same play on 5 jour 2 premiers jours
* F (L) play-id, min(ev-date) as date F Acti GB play-id as a
F Acti as b ON a.play-id = b.play-id AND a.ev-date + 1 = b.ev-date
count apple chest
sum Cb.app-c + CASE WHEN c.app-c = NULL THEN 0 ELSE c.app-c
END as app-c F Boxes as b Left J Chest as c ON b.ch-id = c.ch-id
Warehouse manager (vol)
name as wn, sum (width * len * hei * units) as vol F Warehouse
Prod ON pr-id GB name

avg time process machine
a.mach-id, AVG (b.time - a.time) as proc-time F act a J
act as b ON a.mach-id = b.mach-id AND a.proc-id = b.proc-id
W a.act-type = 'stat' b = 'end' GB a.mach-id
Second highest salary
Name(salary) as SndHigh F Emp W Salary < (S Max(sal) F Emp)
rank scores
S1.Score, (S1.CD Score) F Score as S2 W S2.Score > S1.Score as Rank
F Score as S1 GB S1.Score DESC
tips and users
Request at as Day, R (sum (IF Stat != 'comp', 1, 0) COUNT(Stat), z) as ...
F Trips W Req-at >= '2013-01' AND Req-at <= '2013-01-31' AND CLIENT NOT IN
(S User-id F Users W Barname = 'Yew' AND idem p deliver GB Req-
at as Day)
a.play-id, a.ev-date, sum (b.gamesplayed) F acti a JOIN act h ON id
AND ev-date >= b.ev-date GB a.pl-id, b.ev-date
first 2nd day played
C(*) b.pl-id C(*) a.pl-id F (L) pl-id, MIN(ev-date) F Acti GB id a L Act b
ON a.id = b.id AND e.date = 1 = b.date
human traffic stadium - consecutive date
S1.S1.F1 S2.S2 S3 S4 S5 S6 S7 S8 S9 S10 S11 S12 S13 S14 S15 S16 S17 S18 S19 S20 S21 S22 S23 S24 S25 S26 S27 S28 S29 S30 S31 S32 S33 S34 S35 S36 S37 S38 S39 S40 S41 S42 S43 S44 S45 S46 S47 S48 S49 S50 S51 S52 S53 S54 S55 S56 S57 S58 S59 S60 S61 S62 S63 S64 S65 S66 S67 S68 S69 S70 S71 S72 S73 S74 S75 S76 S77 S78 S79 S80 S81 S82 S83 S84 S85 S86 S87 S88 S89 S90 S91 S92 S93 S94 S95 S96 S97 S98 S99 S100 S101 S102 S103 S104 S105 S106 S107 S108 S109 S110 S111 S112 S113 S114 S115 S116 S117 S118 S119 S120 S121 S122 S123 S124 S125 S126 S127 S128 S129 S130 S131 S132 S133 S134 S135 S136 S137 S138 S139 S140 S141 S142 S143 S144 S145 S146 S147 S148 S149 S150 S151 S152 S153 S154 S155 S156 S157 S158 S159 S160 S161 S162 S163 S164 S165 S166 S167 S168 S169 S170 S171 S172 S173 S174 S175 S176 S177 S178 S179 S180 S181 S182 S183 S184 S185 S186 S187 S188 S189 S190 S191 S192 S193 S194 S195 S196 S197 S198 S199 S200 S201 S202 S203 S204 S205 S206 S207 S208 S209 S210 S211 S212 S213 S214 S215 S216 S217 S218 S219 S220 S221 S222 S223 S224 S225 S226 S227 S228 S229 S230 S231 S232 S233 S234 S235 S236 S237 S238 S239 S240 S241 S242 S243 S244 S245 S246 S247 S248 S249 S250 S251 S252 S253 S254 S255 S256 S257 S258 S259 S260 S261 S262 S263 S264 S265 S266 S267 S268 S269 S270 S271 S272 S273 S274 S275 S276 S277 S278 S279 S280 S281 S282 S283 S284 S285 S286 S287 S288 S289 S290 S291 S292 S293 S294 S295 S296 S297 S298 S299 S300 S301 S302 S303 S304 S305 S306 S307 S308 S309 S310 S311 S312 S313 S314 S315 S316 S317 S318 S319 S320 S321 S322 S323 S324 S325 S326 S327 S328 S329 S330 S331 S332 S333 S334 S335 S336 S337 S338 S339 S340 S341 S342 S343 S344 S345 S346 S347 S348 S349 S350 S351 S352 S353 S354 S355 S356 S357 S358 S359 S360 S361 S362 S363 S364 S365 S366 S367 S368 S369 S370 S371 S372 S373 S374 S375 S376 S377 S378 S379 S380 S381 S382 S383 S384 S385 S386 S387 S388 S389 S390 S391 S392 S393 S394 S395 S396 S397 S398 S399 S400 S401 S402 S403 S404 S405 S406 S407 S408 S409 S410 S411 S412 S413 S414 S415 S416 S417 S418 S419 S420 S421 S422 S423 S424 S425 S426 S427 S428 S429 S430 S431 S432 S433 S434 S435 S436 S437 S438 S439 S440 S441 S442 S443 S444 S445 S446 S447 S448 S449 S450 S451 S452 S453 S454 S455 S456 S457 S458 S459 S460 S461 S462 S463 S464 S465 S466 S467 S468 S469 S470 S471 S472 S473 S474 S475 S476 S477 S478 S479 S480 S481 S482 S483 S484 S485 S486 S487 S488 S489 S490 S491 S492 S493 S494 S495 S496 S497 S498 S499 S500 S501 S502 S503 S504 S505 S506 S507 S508 S509 S510 S511 S512 S513 S514 S515 S516 S517 S518 S519 S520 S521 S522 S523 S524 S525 S526 S527 S528 S529 S530 S531 S532 S533 S534 S535 S536 S537 S538 S539 S540 S541 S542 S543 S544 S545 S546 S547 S548 S549 S550 S551 S552 S553 S554 S555 S556 S557 S558 S559 S560 S561 S562 S563 S564 S565 S566 S567 S568 S569 S570 S571 S572 S573 S574 S575 S576 S577 S578 S579 S580 S581 S582 S583 S584 S585 S586 S587 S588 S589 S590 S591 S592 S593 S594 S595 S596 S597 S598 S599 S600 S601 S602 S603 S604 S605 S606 S607 S608 S609 S610 S611 S612 S613 S614 S615 S616 S617 S618 S619 S620 S621 S622 S623 S624 S625 S626 S627 S628 S629 S630 S631 S632 S633 S634 S635 S636 S637 S638 S639 S640 S641 S642 S643 S644 S645 S646 S647 S648 S649 S650 S651 S652 S653 S654 S655 S656 S657 S658 S659 S660 S661 S662 S663 S664 S665 S666 S667 S668 S669 S670 S671 S672 S673 S674 S675 S676 S677 S678 S679 S680 S681 S682 S683 S684 S685 S686 S687 S688 S689 S690 S691 S692 S693 S694 S695 S696 S697 S698 S699 S700 S701 S702 S703 S704 S705 S706 S707 S708 S709 S710 S711 S712 S713 S714 S715 S716 S717 S718 S719 S720 S721 S722 S723 S724 S725 S726 S727 S728 S729 S730 S731 S732 S733 S734 S735 S736 S737 S738 S739 S740 S741 S742 S743 S744 S745 S746 S747 S748 S749 S750 S751 S752 S753 S754 S755 S756 S757 S758 S759 S760 S761 S762 S763 S764 S765 S766 S767 S768 S769 S770 S771 S772 S773 S774 S775 S776 S777 S778 S779 S780 S781 S782 S783 S784 S785 S786 S787 S788 S789 S790 S791 S792 S793 S794 S795 S796 S797 S798 S799 S800 S801 S802 S803 S804 S805 S806 S807 S808 S809 S810 S811 S812 S813 S814 S815 S816 S817 S818 S819 S820 S821 S822 S823 S824 S825 S826 S827 S828 S829 S830 S831 S832 S833 S834 S835 S836 S837 S838 S839 S840 S841 S842 S843 S844 S845 S846 S847 S848 S849 S850 S851 S852 S853 S854 S855 S856 S857 S858 S859 S860 S861 S862 S863 S864 S865 S866 S867 S868 S869 S870 S871 S872 S873 S874 S875 S876 S877 S878 S879 S880 S881 S882 S883 S884 S885 S886 S887 S888 S889 S890 S891 S892 S893 S894 S895 S896 S897 S898 S899 S900 S901 S902 S903 S904 S905 S906 S907 S908 S909 S910 S911 S912 S913 S914 S915 S916 S917 S918 S919 S920 S921 S922 S923 S924 S925 S926 S927 S928 S929 S930 S931 S932 S933 S934 S935 S936 S937 S938 S939 S940 S941 S942 S943 S944 S945 S946 S947 S948 S949 S950 S951 S952 S953 S954 S955 S956 S957 S958 S959 S960 S961 S962 S963 S964 S965 S966 S967 S968 S969 S970 S971 S972 S973 S974 S975 S976 S977 S978 S979 S980 S981 S982 S983 S984 S985 S986 S987 S988 S989 S990 S991 S992 S993 S994 S995 S996 S997 S998 S999 S1000 S1001 S1002 S1003 S1004 S1005 S1006 S1007 S1008 S1009 S1010 S1011 S1012 S1013 S1014 S1015 S1016 S1017 S1018 S1019 S1020 S1021 S1022 S1023 S1024 S1025 S1026 S1027 S1028 S1029 S1030 S1031 S1032 S1033 S1034 S1035 S1036 S1037 S1038 S1039 S1040 S1041 S1042 S1043 S1044 S1045 S1046 S1047 S1048 S1049 S1050 S1051 S1052 S1053 S1054 S1055 S1056 S1057 S1058 S1059 S1060 S1061 S1062 S1063 S1064 S1065 S1066 S1067 S1068 S1069 S1070 S1071 S1072 S1073 S1074 S1075 S1076 S1077 S1078 S1079 S1080 S1081 S1082 S1083 S1084 S1085 S1086 S1087 S1088 S1089 S1090 S1091 S1092 S1093 S1094 S1095 S1096 S1097 S1098 S1099 S1100 S1101 S1102 S1103 S1104 S1105 S1106 S1107 S1108 S1109 S1110 S1111 S1112 S1113 S1114 S1115 S1116 S1117 S1118 S1119 S1120 S1121 S1122 S1123 S1124 S1125 S1126 S1127 S1128 S1129 S1130 S1131 S1132 S1133 S1134 S1135 S1136 S1137 S1138 S1139 S1140 S1141 S1142 S1143 S1144 S1145 S1146 S1147 S1148 S1149 S1150 S1151 S1152 S1153 S1154 S1155 S1156 S1157 S1158 S1159 S1160 S1161 S1162 S1163 S1164 S1165 S1166 S1167 S1168 S1169 S1170 S1171 S1172 S1173 S1174 S1175 S1176 S1177 S1178 S1179 S1180 S1181 S1182 S1183 S1184 S1185 S1186 S1187 S1188 S1189 S1190 S1191 S1192 S1193 S1194 S1195 S1196 S1197 S1198 S1199 S1200 S1201 S1202 S1203 S1204 S1205 S1206 S1207 S1208 S1209 S1210 S1211 S1212 S1213 S1214 S1215 S1216 S1217 S1218 S1219 S1220 S1221 S1222 S1223 S1224 S1225 S1226 S1227 S1228 S1229 S1230 S1231 S1232 S1233 S1234 S1235 S1236 S1237 S1238 S1239 S1240 S1241 S1242 S1243 S1244 S1245 S1246 S1247 S1248 S1249 S1250 S1251 S1252 S1253 S1254 S1255 S1256 S1257 S1258 S1259 S1260 S1261 S1262 S1263 S1264 S1265 S1266 S1267 S1268 S1269 S1270 S1271 S1272 S1273 S1274 S1275 S1276 S1277 S1278 S1279 S1280 S1281 S1282 S1283 S1284 S1285 S1286 S1287 S1288 S1289 S1290 S1291 S1292 S1293 S1294 S1295 S1296 S1297 S1298 S1299 S1300 S1301 S1302 S1303 S1304 S1305 S1306 S1307 S1308 S1309 S1310 S1311 S1312 S1313 S1314 S1315 S1316 S1317 S1318 S1319 S1320 S1321 S1322 S1323 S1324 S1325 S1326 S1327 S1328 S1329 S1330 S1331 S1332 S1333 S1334 S1335 S1336 S1337 S1338 S1339 S1340 S1341 S1342 S1343 S1344 S1345 S1346 S1347 S1348 S1349 S1350 S1351 S1352 S1353 S1354 S1355 S1356 S1357 S1358 S1359 S1360 S1361 S1362 S1363 S1364 S1365 S1366 S1367 S1368 S1369 S1370 S1371 S1372 S1373 S1374 S1375 S1376 S1377 S1378 S1379 S1380 S1381 S1382 S1383 S1384 S1385 S1386 S1387 S1388 S1389 S1390 S1391 S1392 S1393 S1394 S1395 S1396 S1397 S1398 S1399 S1400 S1401 S1402 S1403 S1404 S1405 S1406 S1407 S1408 S1409 S1410 S1411 S1412 S1413 S1414 S1415 S1416 S1417 S1418 S1419 S1420 S1421 S1422 S1423 S1424 S1425 S1426 S1427 S1428 S1429 S1430 S1431 S1432 S1433 S1434 S1435 S1436 S1437 S1438 S1439 S1440 S1441 S1442 S1443 S1444 S1445 S1446 S1447 S1448 S1449 S1450 S1451 S1452 S1453 S1454 S1455 S1456 S1457 S1458 S1459 S1460 S1461 S1462 S1463 S1464 S1465 S1466 S1467 S1468 S1469 S1470 S1471 S1472 S1473 S1474 S1475 S1476 S1477 S1478 S1479 S1480 S1481 S1482 S1483 S1484 S1485 S1486 S1487 S1488 S1489 S1490 S1491 S1492 S1493 S1494 S1495 S1496 S1497 S1498 S1499 S1500 S1501 S1502 S1503 S1504 S1505 S1506 S1507 S1508 S1509 S1510 S1511 S1512 S1513 S1514 S1515 S1516 S1517 S1518 S1519 S1520 S1521 S1522 S1523 S1524 S1525 S1526 S1527 S1528 S1529 S1530 S1531 S1532 S1533 S1534 S1535 S1536 S1537 S1538 S1539 S1540 S1541 S1542 S1543 S1544 S1545 S1546 S1547 S1548 S1549 S1550 S1551 S1552 S1553 S1554 S1555 S1556 S1557 S1558 S1559 S1560 S1561 S1562 S1563 S1564 S1565 S1566 S1567 S1568 S1569 S1570 S1571 S1572 S1573 S1574 S1575 S1576 S1577 S1578 S1579 S1580 S1581 S1582 S1583 S1584 S1585 S1586 S1587 S1588 S1589 S1590 S1591 S1592 S1593 S1594 S1595 S1596 S1597 S1598 S1599 S1600 S1601 S1602 S1603 S1604 S1605 S1606 S1607 S1608 S1609 S1610 S1611 S1612 S1613 S1614 S1615 S1616 S1617 S1618 S1619 S1620 S1621 S1622 S1623 S1624 S1625 S1626 S1627 S1628 S1629 S1630 S1631 S1632 S1633 S1634 S1635 S1636 S1637 S1638 S1639 S1640 S1641 S1642 S1643 S1644 S1645 S1646 S1647 S1648 S1649 S1650 S1651 S1652 S1653 S1654 S1655 S1656 S1657 S1658 S1659 S1660 S1661 S1662 S1663 S1664 S1665 S1666 S1667 S1668 S1669 S1670 S1671 S1672 S1673 S1674 S1675 S1676 S1677 S1678 S1679 S1680 S1681 S1682 S1683 S1684 S1685 S1686 S1687 S1688 S1689 S1690 S1691 S1692 S1693 S1694 S1695 S1696 S1697 S1698 S1699 S1700 S1701 S1702 S1703 S1704 S1705 S1706 S1707 S1708 S1709 S1710 S1711 S1712 S1713 S1714 S1715 S1716 S1717 S1718 S1719 S1720 S1721 S1722 S1723 S1724 S1725 S1726 S1727 S1728 S1729 S1730 S1731 S1732 S1733 S1734 S1735 S1736 S1737 S1738 S1739 S1740 S1741 S1742 S1743 S1744 S1745 S1746 S1747 S1748 S1749 S1750 S1751 S1752 S1753 S1754 S1755 S1756 S1757 S1758 S1759 S1760 S1761 S1762 S1763 S1764 S1765 S1766 S1767 S1768 S1769 S1770 S1771 S1772 S1773 S1774 S1775 S1776 S1777 S1778 S1779 S1780 S1781 S1782 S1783 S1784 S1785 S1786 S1787 S1788 S1789 S1790 S1791 S1792 S1793 S1794 S1795 S1796 S1797 S1798 S1799 S1800 S1801 S1802 S1803 S1804 S1805 S1806 S1807 S1808 S1809 S1810 S1811 S1812 S1813 S1814 S1815 S1816 S1817 S1818 S1819 S1820 S1821 S1822 S1823 S1824 S1825 S1826 S1827 S1828 S1829 S1830 S1831 S1832 S1833 S1834 S1835 S1836 S1837 S1838 S1839 S1840 S1841 S1842 S1843 S1844 S1845 S1846 S1847 S1848 S1849 S1850 S1851 S1852 S1853 S1854 S1855 S1856 S1857 S1858 S1859 S1860 S1861 S1862 S1863 S1864 S1865 S1866 S1867 S1868 S1869 S1870 S1871 S1872 S1873 S1874 S1875 S1876 S1877 S1878 S1879 S1880 S1881 S1882 S1883 S1884 S1885 S1886 S1887 S1888 S1889 S1890 S1891 S1892 S1893 S1894 S1895 S1896 S1897 S1898 S1899 S1900 S1901 S1902 S1903 S1904 S1905 S1906 S1907 S1908 S1909 S1910 S1911 S1912 S1913 S1914 S1915 S1916 S1917 S1918 S1919 S1920 S1921 S1922 S1923 S1924 S1925 S1926 S1927 S1928 S1929 S1930 S1931 S1932 S1933 S1934 S1935 S1936 S1937 S1938 S1939 S1940 S1941 S1942 S1943 S1944 S1945 S1946 S1947 S1948 S1949 S1950 S1951 S1952 S1953 S1954 S1955 S1956 S1957 S1958 S1959 S1960 S1961 S1962 S1963 S1964 S1965 S1966 S1967 S1968 S1969 S1970 S1971 S1972 S1973 S1974 S1975 S1976 S1977 S1978 S1979 S1980 S1981 S1982 S1983 S1984 S1985 S1986 S1987 S1988 S1989 S1990 S1991 S1992 S1993 S1994 S1995 S1996 S1997 S1998 S1999 S2000 S2001 S2002 S2003 S2004 S2005 S2006 S2007 S2008 S2009 S2010 S2011 S2012 S2013 S2014 S2015 S2016 S2017 S2018 S2019 S2020 S2021 S2022 S2023 S2024 S2025 S2026 S2027 S2028 S2029 S2030 S2031 S2032 S2033 S2034 S2035 S2036 S2037 S2038 S2039 S2040 S2041 S2042 S2043 S2044 S2045 S2046 S2047 S2048 S2049 S2050 S2051 S2052 S2053 S2054 S2055 S2056 S2057 S2058 S2059 S2060 S2061 S2062 S2063 S2064 S2065 S2066 S2067 S2068 S2069 S2070 S2071 S2072 S2073 S2074 S2075 S2076 S2077 S2078 S2079 S2080 S2081 S2082 S2083 S2084 S2085 S2086 S2087 S2088 S2089 S2090 S2091 S2092 S2093 S2094 S2095 S2096 S2097 S2098 S2099 S2100 S2101 S2102 S2103 S2104 S2105 S2106 S2107 S2108 S2109 S2110 S2111 S2112 S2113 S2114 S2115 S2116 S2117 S2118 S2119 S2120 S2121 S2122 S2123 S2124 S2125 S2126 S2127 S2128 S2129 S2130 S2131 S2132 S2133 S2134 S2135 S2136 S2137 S2138 S2139 S2140 S2141 S2142 S2143 S2144 S2145 S2146 S2147 S2148 S2149 S2150 S2151 S2152 S2153 S2154 S2155 S2156 S2157 S2158 S2159 S2160 S2161 S2162 S2163 S2164 S2165 S2166 S2167 S2168 S2169 S2170 S2171 S2172 S2173 S2174 S2175 S2176 S2177 S2178 S2179 S2180 S2181 S2182 S2183 S2184 S2185 S2186 S2187 S2188 S2189 S2190 S2191 S2192 S2193 S2194 S2195 S2196 S2197 S2198 S2199 S2200 S2201 S2202 S2203 S2204 S2205 S2206 S2207 S2208 S2209 S2210 S2211 S2212 S2213 S2214 S2215 S2216 S2217 S2218 S2219 S2220 S2221 S2222 S2223 S2224 S2225 S2226 S2227 S2228 S2229 S2230 S2231 S2232 S2233 S2234 S2235 S2236 S2237 S2238 S2239 S2240 S2241 S2242 S2243 S2244 S2245 S2246 S2247 S2248 S2249 S2250 S2251 S2252 S2253 S2254 S2255 S2256 S2257 S2258 S2259 S2260 S2261 S2262 S2263 S2264 S2265 S2266 S2267 S2268 S2269 S2270 S2271 S2272 S2273 S2274 S2275 S2276 S2277 S2278 S2279 S2280 S2281 S2282 S2283 S2284 S2285 S2286 S2287 S2288 S2289 S2290 S2291 S2292 S2293 S2294 S2295 S2296 S2297 S2298 S2299 S2300 S2301 S2302 S2303 S2304 S2305 S2306 S2307 S2308 S2309 S2310 S2311 S2312 S2313 S2314 S2315 S2316 S2317 S2318 S2319 S2320 S2321 S2322 S2323 S2324 S2325 S2326 S2327 S2328 S2329 S2330 S2331 S2332 S2333 S2334 S2335 S2336 S2337 S2338 S2339 S2340 S2341 S2342 S2343 S2344 S2345 S2346 S2347 S2348 S2349 S2350 S2351 S2352 S2353 S2354 S2355 S2356 S2357 S2358 S2359 S2360 S2361 S2362 S2363 S2364 S2365 S2366 S2367 S2368 S2369 S2370 S2371 S2372 S2373 S2374 S2375 S2376 S2377 S2378 S2379 S2380 S2381 S2382 S2383 S2384 S2385 S2386 S2387 S2388 S2389 S2390 S2391 S2392 S2393 S2394 S2395 S2396 S2397 S2398 S2399 S2400 S2401 S2402 S2403 S2404 S2405 S2406 S2407 S2408 S2409 S2410 S2411 S2412 S2413 S2414 S2415 S2416 S2417 S2418 S2419 S2420 S2421 S2422 S2423 S2424 S2425 S2426 S2427 S2428 S2429 S2430 S2431 S2432 S2433 S2434 S2435 S2436 S2437 S2438 S2439 S2440 S2441 S2442 S2443 S2444 S2445 S2446 S2447 S2448 S2449 S2450 S2451 S2452 S2453 S2454 S2455 S2456 S2457 S2458 S2459 S2460 S2461 S2462 S2463 S2464 S2465 S2466 S2467 S2468 S2469 S2470 S2471 S2472 S2473 S2474 S2475 S2476 S2477 S2478 S2479 S2480 S2481 S2482 S2483 S2484 S2485 S2486 S2487 S2488 S2489 S2490 S2491 S2492 S2493 S2494 S2495 S2496 S2497 S2498 S2499 S2500 S2501 S2502 S2503 S2504 S2505 S2506 S2507 S2508 S2509 S2510 S2511 S2512 S2513 S2514 S2515 S2516 S2

WINDOWS → permet de **OB** pas que de tableau de source mais on s'inscrit dans une requête sans avoir à écrire chaque ligne une agg^o sur des grp sans avoir à les regg^{er}.
 ens de lignes ordonnées pour chaque ligne des table de sortie
 ds **S** (on chq ligne) et pr agg^o sur ses windows pour (**OB** de wind)
PB on chq ligne i, aj^t à son wind toutes les lignes du output avec m (dept-name)
RB opac 1 fol pour chq ligne i, ne garde que i et sa suivante (i & i+1)
 si pas précisé : **RB UNBOUND P AND CURRENT ROW**

S dept-name, avg(sal), avg(avg(sal)) **OVER** w **F** teacher **GB** dept-name **WI** w **AS** (**OB** dept-name **RB 1P, 1F**);
 ↳ de 1 ligne par dept ↳ moy de moy préc moy des moy suiv.

S name, **MAX** sal **OVER** (**OB** sal) **F** teacher → à chq ligne
 ↳ qd est max salaire cumulé jusqu'à présent? = max cumulative
 ↳ ne chq rien car **OB** donne ordonné du + petit au + gd
 ↳ toujours le même sal (le + petit) car ordonné du + petit au + gd
 ↳ donc le + petit sal déjà cumulé = le 1^{er}
 ↳ moy cumulative.

RANK : si m val alors m rang
S **RANK** (**OVER** (**OB** salary **DESC**) **F** teacher
 ↳ la colonne sur laq^l on fait **RANK** n'apparaît pas si on la demande.
 donc **OB** du rank ≠ **OB** de la requête.
DENSERANK : rank mais sans trou : 1 2 3 4 5 6 7
ROW-NB (**OVER**) → ni trou ni m rang 1 2 3 4 5 6 7 8
 ↳ ignore **RB**
NTILE(n) **OVER** div en n parts égales
 Rank : **OB** obligé!

LAG(attrib, num) **OVER** w prend val de col attrib qui est n lignes + haute.
LEAD : **name** **lag** **lead** **name**, 2
 ↳ **nth-value**(name, 2)
 ↳ A n'a rien car on a pas précisé **RB** de rep l^{re} ligne courante + au dessus (pas après de de p avoir courat de N-Val fait pour A y a pas de n=2)
 ↳ avec **RB UNBOUND P AND UNB FOL**

salaire actuel - salaire préc
S name, sal - **LAG**(sal, 1) **OVER** w
PARTITION BY dept-name → fait mini table selon dept-name
 on chq ligne on fait **RANK** ds pour sa mini table
 donne nom teacher avec rang salaire selon dept :
S name, dept-name **RC** **OVER** (**PB** dept-name **OB** salary) **F** teacher **OB** dept-name
 ↳ qd on fait **RB-PAND-F** se fait au sein des partitions. (unbound = pas H table mais tout grp.)

ROWS : **RB** unbound **P** AND unbound **F** EXCLUDE current row
RANGE : en b^o du **OB** et non positif
S name, sal **OVER** (**PB** dept-name **OB** salary **RANGE** 0 **P** AND 5 **F** teacher);
 = compte tous les profs dont salaire est mensal - 15 et mensal + 5
 ↳ si range de pd de **OB** il en faut 1 seul
OB sal, name ⇒ erreur

GROUPS : c^o nous mais avec 0 de lignes si m valeurs (m si au milieu!)
BETWEEN **S** name, sal **AVG**(sal), **OVER** w
 ↳ **need** **OB** **F** teacher **WI** **AS** (**OB** sal **GROUPS** 0 **P** AND 1 **F**);
 = **ROWS** si H est distinct
 ↳ ex sal^{re} moy dep
 ↳ qd on fait **GB** window n'aura plus accès à toute la table! (après **GB**)
 de fait cohérence entre **GB** et les rank/range/groups.

si on veut qu'une ligne par dep ⇒ **GROUP BY**
 (donc on efface des noms de)
 si on veut garder nom + salaire de chq prof **ET** sal moy par dep ⇒ **PARTITION BY**

pour chaque étudiant et dis cours qu'il a suivi, affiche son ID, year et nb cred du cours.
 Et calc + affiche en respect ordre chron des cours: * rang du cours dans le parcours de l'etu
 * tot cumule des crédits acquis jusqu'à ce cours
 * moy mobile du nb de cred sur les 3 derniers cours (2 prec et act)

S. id, t. year, c. credits, **RANK** w-base **AS** w-base **AS** running-tot
SUM (c. credits) **AS** w-base **AS** running-tot
AVG (c. credits) **AS** w-mobite **AS** avg-mob

F student **AS** s **F** takes **AS** t **ON** s.id = t.id **F** courses **AS** c **ON** t. course-id = c. course-id
W w-base **AS** (PB s.id **OB** t. year, t. semester), w-mobite **AS** (w-base **AS** 2 prec **AND** current R)

RECURSION

WITH RECURSIVE graph-edges (node-f, node-to, cost) **AS** (Values ('A', 'B', 1), ('A', 'C', 1), ('B', 'D', 1), ('C', 'D', 1), ('C', 'E', 1), ('E', 'F', 1)),
 graph-paths **AS** (S node-f, node-t, ARRAY[n-f, n-t] **AS** path, 1 **AS** depth, cost
F graph-edges **W** node-f = 'A' # 1^{ere} ligne
 # 2^{eme} ligne
 # 3^{eme} ligne
 # 4^{eme} ligne
 # 5^{eme} ligne
 # 6^{eme} ligne
 # 7^{eme} ligne
 # 8^{eme} ligne
 # 9^{eme} ligne
 # 10^{eme} ligne
 # 11^{eme} ligne
 # 12^{eme} ligne
 # 13^{eme} ligne
 # 14^{eme} ligne
 # 15^{eme} ligne
 # 16^{eme} ligne
 # 17^{eme} ligne
 # 18^{eme} ligne
 # 19^{eme} ligne
 # 20^{eme} ligne
 # 21^{eme} ligne
 # 22^{eme} ligne
 # 23^{eme} ligne
 # 24^{eme} ligne
 # 25^{eme} ligne
 # 26^{eme} ligne
 # 27^{eme} ligne
 # 28^{eme} ligne
 # 29^{eme} ligne
 # 30^{eme} ligne
 # 31^{eme} ligne
 # 32^{eme} ligne
 # 33^{eme} ligne
 # 34^{eme} ligne
 # 35^{eme} ligne
 # 36^{eme} ligne
 # 37^{eme} ligne
 # 38^{eme} ligne
 # 39^{eme} ligne
 # 40^{eme} ligne
 # 41^{eme} ligne
 # 42^{eme} ligne
 # 43^{eme} ligne
 # 44^{eme} ligne
 # 45^{eme} ligne
 # 46^{eme} ligne
 # 47^{eme} ligne
 # 48^{eme} ligne
 # 49^{eme} ligne
 # 50^{eme} ligne
 # 51^{eme} ligne
 # 52^{eme} ligne
 # 53^{eme} ligne
 # 54^{eme} ligne
 # 55^{eme} ligne
 # 56^{eme} ligne
 # 57^{eme} ligne
 # 58^{eme} ligne
 # 59^{eme} ligne
 # 60^{eme} ligne
 # 61^{eme} ligne
 # 62^{eme} ligne
 # 63^{eme} ligne
 # 64^{eme} ligne
 # 65^{eme} ligne
 # 66^{eme} ligne
 # 67^{eme} ligne
 # 68^{eme} ligne
 # 69^{eme} ligne
 # 70^{eme} ligne
 # 71^{eme} ligne
 # 72^{eme} ligne
 # 73^{eme} ligne
 # 74^{eme} ligne
 # 75^{eme} ligne
 # 76^{eme} ligne
 # 77^{eme} ligne
 # 78^{eme} ligne
 # 79^{eme} ligne
 # 80^{eme} ligne
 # 81^{eme} ligne
 # 82^{eme} ligne
 # 83^{eme} ligne
 # 84^{eme} ligne
 # 85^{eme} ligne
 # 86^{eme} ligne
 # 87^{eme} ligne
 # 88^{eme} ligne
 # 89^{eme} ligne
 # 90^{eme} ligne
 # 91^{eme} ligne
 # 92^{eme} ligne
 # 93^{eme} ligne
 # 94^{eme} ligne
 # 95^{eme} ligne
 # 96^{eme} ligne
 # 97^{eme} ligne
 # 98^{eme} ligne
 # 99^{eme} ligne
 # 100^{eme} ligne

PL/PGSQL

decla ° avant begin

BEGIN **IF** **THEN** **ELSEIF** **THEN** **ELSE** **END IF** **RETURN** une ligne **END**;

CREATE OR REPLACE FUNCTION nom-f (nom-arg TYPE) **RETURNS** TYPE **AS** \$\$
DECLARE # decla des variables
 nom1 TYPE;
BEGIN
 S a, b **INTO** nom1, - **F** col **W** col.nomme = nom-f.nomme;
 result = result || concat
RETURN result;
END;
 \$\$ LANGUAGE plpgsql;

CORF fun (nom1 VARCHAR(50))
RET SETOF teaches **AS** \$\$
DECL teach-id VAR(5)
BEG **FOREACH** t **IN** ARRAY nom1
LOOP
RET QUERY
 S **FW** ID = t-id;
ENDLOOP
 \$\$ LANGUAGE plpgsql;

CORF fun (arg TYPE)
RET NOM **AS** \$\$
DECL a, b;
BEGIN
FOR bidule **IN** :
 S **FW** (NOT NULL)
LOOP
CASE bidule
 WHEN THEN a =
 ELSE
ENDCASE
IF a NOT NULL **THEN**
 b =
ENDLOOP

TRIGGER

CORF ma-f-trigg()
RET TRIGGER **AS** \$\$
BEGIN
 cas 1: avant insert
IF NEW.col IS NULL **THEN**
 NEW.col := 0; val del
ENDIF
 cas 2: comp OLD/NEW
IF OLD.col > NEW.col **THEN**
RAISE EXC 'Interdit de ↓'
ENDIF
 cas 3: règle Unit
IF NEW.col < 30 **THEN**
RAISE EXC 'Val trop ↓'
ELSEIF NEW.col > 50 **THEN**
RAISE EXC 'Val trop ↑'
ENDIF
RET NEW;
END;
 \$\$ LANGUAGE plpgsql;

CREATE TRIGGER ta
BEFORE **INSERT** **ON** tab **FOR EACH ROW** **EXECUTE FUNCTION** ma-f-trigg();
 cas 5: avant insert global
IF (S 1 F) dep W budget > 10 **THEN**
RAISE EXC 'Attention! dép de budget!';
ENDIF
RET NULL;

BEFORE UPDATE ON tab
BEF INS OR UPD ON tab

AND, OR, NOT dans le WHERE

distinct dans le SELECT

avg, sum, max, min AGGREGATIONS
count

Dans le SELECT on peut

CONCAT, salary * 2, AS

IN, NOT IN dans le WHERE

ex: WHERE dept.name NOT IN ('Comp.Sci.', 'History')

HAVING pour les aggregations

↳ souvent avec GROUP BY

Sous requête: toujours renommée

WITH: table temporaire: ↓

OPERATIONS: UNION, UNION ALL for dupes

INTERSECT, INTERSECT ALL keep dupes

EXCEPT, EXCEPT ALL keep dupes

STRING, VARCHAR, WHERE name LIKE

ex: '%an%', ('__an%')

OPERATIONS avec null → null

AGGREGATIONS IGNORE null

grade ~~<~~ null grade IS NULL

salary ~~<~~ null salary IS NOT NULL

ex: SELECT (grade IS NULL) FROM takers

retourne tableau true et false

avec null retourne un bool

ILL ex: salary > ALL (table)

ou: au moins un

EXIST vérifie présence dans la table

CASE

WHEN condition THEN output

WHEN

ELSE

AGGREGATION avec CASE

ex: AVG (CASE

THEN 4.0

END THEN 1.0)

- INNER JOIN - on peut combiner
avec AND, OR

- CROSS JOIN - comme produit
x de ON

- LEFT OUTER JOIN - ON
normalise un join classique

• ajoute les lignes unmatched à
gauche et remplit à droite avec
null.

- RIGHT OUTER JOIN - ON miroir

- NATURAL JOIN -
x de ON, utilise la primary key

USING pour matcher sans égal
() obligatoire

INSERT INTO

DELETE FROM - WHERE

Primary key ne peut avoir de null

DEFAULT: pour éviter null comme def

CHECK contrainte

CASCADE

DEPENDENCY check redondance

1NF - pas de listes, tableau et
valeurs uniques

2NF - 1NF

- tout élément non clé
depend de la clé

3NF - 2NF

- aucune dépendance transitive

ex: A → B et B → C, A → C dépendance transitive

Solution: séparer en 2 tables

BCNF (Boyce-Codd Normal Form)

∀ x → y, x clé candidate

ex:

Prof	Matière	Salle
Dupont	Math	A1
Dupont	Phys	A2

Suppose

• Prof → Salle

• (Prof, Matière) clé primaire

Prof tt seul pas clé candidate mais détermine salle

Donc séparer

- Version plus stricte que 3NF

Aucune redondance possible

comb de personnes fit, pour 200 us

```
WITH total_weight (ordering, numweight) AS (
  SELECT A.ordering, sum(B.weight)
  FROM stream AS A INNER JOIN stream AS B
  ON B.ordering <= A.ordering)

```

tableau comm
on ordre

```
GROUP BY A.ordering)
SELECT min(ordering)
FROM total_weight
WHERE numweight <= 200;
```

match (host-id int, host-goals int, guest-id int, guest-goals int)
:older par team et points

with mytable (id, points) AS (

```
( SELECT host-id, (CASE
  WHEN > 3 THEN 3
  ELSE 0 END)
  FROM match)
UNION ALL
( SELECT
  FROM match))
```

table avec
les points
gagner par
match.

Union all
car on veut

redondance

```
SELECT id, sum(points)
FROM mytable
GROUP BY id;
```

suite la plus longue

with pairs (id1, id2, id3) AS (

```
SELECT distinct A.id, B.id, C.id
FROM number AS A, number AS B, number AS C
WHERE A.id <= B.id AND B.id <= C.id,
```

tableau avec
 $a \leq b \leq c$

pairs - total (id1, id2, total) AS (

```
SELECT id1, id2, count(C)
```

```
FROM pairs
```

```
GROUP BY id1, id2
```

```
HAVING count(C) = id2 - id1 + 1)
```

on vérifie si les
nombres sont à la suite
et si oui garder la taille
de la suite

```
SELECT A.id1 AS id_start, A.id2 AS id_end
```

```
FROM pairs - total AS A
```

```
WHERE A.total >= ALL (SELECT total FROM pairs - total AS B)
ORDER BY A.id1
```

on prend le max
avec ALL on peut
en prendre plusieurs

R(abcde)

CD → B

A → D

E → C

AE → AB

the output table.

- Example:

FROM teacher

Window WAS (size of Rows Between $\frac{i-1}{i-1}$ AND $\frac{i+1}{i+1}$)

② select name, salary, Avg (salary) ^{unbounded} over w

FROM teacher

FROM teacher
Window w AS (Partition by deptname

Rows Between unBounded Preading

Partition By: Prends les lignes avec les même dept-name (dans ce cas)

Stacking functions:

- lag (attrib, num): gets value of attrib in num row earlier in the window
- lead (attrib, num): gets value of attrib in num row later in the window.
- NTH value (attrib, num) gets value attrib at num row;

```
select name, lag(name, 2) over w
from teacher
window w as (order by salary);
```

id	name	log
11111	MoZart	
32343	EL scid	
58583	calibri	MoZart
10101	Spring	EL scid

Recursion:

WITH RECURSIVE fact AS
(Select 1 as n, 1 as facto
Union ALL

Union ALL
Select n+1, facto * (n+1)

From fact fact
Where $n < 5$)

n

Select *
FROM fact;

Case:

Case A When B Then C

condition pour que $A = B$ on change $A = \mathbb{C}$

Row Number: nombre de la usura (usura $n =$)

Rank () : rang par rapport au over-
ligne final contient n = ligne
Rank () : ligne (fait sans del peut
donner 3)

☞ chiffre consécutif.

Example :

select name, Dense_Rank() over
(ORDER BY dept_name
ASC)

FROM teacher;

window

NILE(N): numéro de groupe forcé
on regroupe les résultats avec un n° de 1 à N
en fonction du bloc window.

Rows unBounded Preceding Volkussi

~~Rangul: regarde dans les cas ~~et~~ entre les conditions et donne les rangs selon cela ainsi que order by.~~

select name, salary, count(*) over w
FROM teacher
Window w AS (order by salary
Range between 1000 and

group: takes order by and (5000);
takes the conditions but if same value
will put it onto the group. (order always by int)

Functions:

Windows W AS (- - -)
W-V AS (- - -)
W-V AS (- - -)

PL / pgSQL:

Create or Replace Function area (radius Numeric)

Returns Numeric As \$\$

DECLASSE

pi Constante Numerică = 3.14159;
 area Numerică;

REGIN

N
area := $\pi \times \text{radius} \times \text{radius}$;

Return area;

END;

Language plays a

 CamScanner

TP10: Rendu @ efficace.

sans requêtes de [S] → Join + [GB]
 [S] d.dept, [S] COUNT(*) → [S] d.dept-n, [C] (s.id) as
 [F] stud → [F] dep d [LJ] stud s
 [W] dep-n=d → [GB] ON dept-n
 [F] dep and. [GB] dept-n.

NOT IN → LEFT JOIN + IS NULL
 [S] s.name [F] stud as s → [S] s.name [F] stud as s
 [W] id NOT IN ([S] t.id [F] teach) → [LJ] takes as t ON id
 [W]HE t.ID IS NULL

JOINTURES implicites (F) ..., ..., (F) [GB] → INDEX sur join
 [S] - T.F (S) - F a, b, c, d → CTE (WITH) de la
 [W] A. = b. → partie lourde
 [GB] - - - → 1) sur les col du [J]
 [W] T.k > 2; → (ou [W] qui join)
 [OB] - - - → 2) sur col qui remplissent col
 → pas de col comp sur la requête calculé
 > 60000
 do [W]
 3) col [GB] et [OB]

CREATE INDEX
 idx - takesid
 ON (course-id, sec-id,
 term, year)

WITH T AS
 la partie lourde
 m'empêcher de requête
 et en supp des
 colonnes pt simplifier

→ do [S] ~ 1 cas
 sans requête scalar
 EXISTS → JOIN @ GB.
 → JOIN @ GB.
 PLSRS AGG° [S] 1 modif → les fonctions
 [S] dept-name (S) AVG(student-c) [S] dept-n,
 [F] (S) [C] * as student-c [C] s.id as st-c
 [F] student as s [W] dept-n = [F] dep [LJ] st
 GB dept-n as sub [ON] dept
 as avg per dept [F] B/ dept.
 [F] dep.
 [S] [C] * [F] [W] ... → [C] (case when
 [I] 1 END)

calculs à # éch⁴ → Over partition by
 étudiants puis grp.
 [S] s.id, s.name, s.dept [S] [C] * [F] takes t [W] id
 [S] [C] * [F] takes t [W] id as tot-cours
 AVG(s.tot-cours) O (PP) s.dept as avgdepcours [F] de
 [S] AVG(tot-cours) [F] student s [LJ] takes t [ON] id
 student s2 [W] s2.dept = s.dept [GB] s.id, t.id, s.name, s.dept, s.tot-cours
 [F] student s;
 as avgdepcours

medium employee salary
 WITH cte AS
 ([S] *, ROW-Numbers() O ([PB] company [OB]
 salary) AS rnk, [C] (id) O ([PB] company)
 as n
 [S] id, company, salary [F] cte Where
 rnk between n/2 AND (n/2)+1.

avg sal dep vs company.
 WITH cte AS ([S] s.empid, s.amount, date-
 format(date) as paymonth, e.dept-id, avg (amount
 format(date)) as comp-avg,
 O ([PB] date format (date)) as comp-avg,
 avg (amount) O ([PB] e.dept-id, date-format
 (pay date)) as dep-avg
 [F] salary [LJ] emp ON s.id = e.id.)
 [S] dep-avg, dep-id, [CASE W] dep-avg
 comp-avg THEN higher [W] ... ELSE lower
 END as comparison [F] cte.

Running Totals with windows
 WITH cte as ([S] *, Max(end-day) O ([PB]
 hall-id [OB] start-day [ROWS BETWEEN]
 unbounded prec and 1 prec) as max-end-day
 [F] hall events),
 cte2 as ([S] [CASE W] start-day > max-end-day
 T 1 ELSE 0 END AS val [F] cte,
 cte3 as ([S] *) SUM (val) O ([PB] hall-id [OB]
 start-day [ROWS BET] unbo prec AND current
 row) as grp [F] cte2.
 [S] hall-id, MIN (start start-day) as start-day
 MAX (end-day) as end-day [F] cte3 [GB]
 hall-id, grp.

Longest winning streak
 WITH cte as ([S] *, ROW-NB() O ([PB] playerid
 [OB] match-day) as match-seq, ROW-NB()
 O ([PB] player-id, result [OB] match-day)
 as res-seq [F] matches),
 cte2 as ([S] play-id, (match seq-res) as
 diff, [C] * as streak, R-NB() O ([PB] playid
 [OB] [C] * DESC) as rnk [F] cte [W] result = 'win'
 [GB] play-id, (match seq res)

[S] [D] m. playid, IFNULL (T.streak, 0) AS
 longest-streak [F] matches as m
 [LJ] ([S] play-id, streak [F] cte2 [W] rnk = 1
 as T [UNION] (play-id);

nb of seniors and juniors to [J] comp.
 WITH cte as ([S] emp-id, exp, sum (sal) O ([PB]
 exp [OB] sal asc, emp-id ASC) as salsum
 [F] candi)
 [S] 'senior' as exp, [C] * as accepted-c [F] de
 Where exp = 'sen' AND salsum <= 70000.

UNION [S] 'jun' as exp [C] *
 WHERE exp = 'jun' and salsum <=
 70000 - IFNULL (MAX (sal sum),
 [S] 70000 - IFNULL (MAX (sal sum),
 [F] de Where exp = 'sen' AND
 sal-sum <= 70000)

OFFSET = tout sauf les n premiers.

nb of transactions per visit

WITH REC cte AS (S) v.visit_date, v.userid,
sum(case w t.amount is null 0 else 1 end)

as num-tran (F) visits v (S) Trans as t (ON)
user-id AND visit date = trans date (GB) visit date,
user-id),

cte 2 as (S) 0 as num-tran (UNION) (S) num-tran + 1

(F) cte 2 (W) num-tran < (S) max (num-tran) (F) cte)

(S) cte 2.num-tran as trans-count, (C) (cte.user-id)

as visit-c (F) cte 2 (S) cte (ON) cte 2.num-tran
= cte.num-tran (GB) cte 2.num-tran (GB) trans-count;

nb services & pay to 3 comp 2.

with cte as (S) *, sum(sal) (O) (PB) emp
(GB) sal) as cum-sal (F) condi),

cte 2 as (S) *, sum(sal) (O) (PB) emp, sal)

as cum-sal-2 (F) cte (W) cum-sal < 70000)

(S) emp-id (F) cte 2 (W) cum-sal-2 < 70000);

1) Faut faire INDEX sur les col concernées

Index useless si trop petit.

Select sur tout le tableau:

Seq scan : tête de tableau.

↳ on a bc

⇒ FROM abc

Output = SELECT
Filter = WHERE
group key = GB
sort key = GB

⇒ Bitmap Index Scan =

⇒ CREATE INDEX

Bitmap Heap Scan
on teacher
↳ FROM teacher.

Index Cond

Nested loop → Outer

Hash Join → "="

output abc

Inner
Join Filter
Hash Cond

→ Seq scan on course c.
out put dep.

→ Hash
materialize
Output : g

→ Seq scan on dep
out put dep
filter: (c.d.buld = 'E')

(S) abc.